ELEC 4260 Project 3 Report

Data-driven Grasp Detection for Robotic Manipulation

Course	ELEC 4260	**Project**	Project 3
Task	RGB-D neural grasp detection, simulation integration, and real-world deployment	**Prepared from**	Final project code, logs, and screenshots

Executive summary

This project implemented a complete RGB-D grasping pipeline: Cornell-dataset training, grasp prediction in DISCOVERSE simulation, hand-eye calibration, and execution on the Airbot arm. RGB-D input clearly outperformed depth-only input. The selected deployed model was the baseline GraspCNN trained with 4-channel RGB-D input, because it offered the best compromise between validation performance, runtime, and deployment stability.

1. Implemented pipeline and code modifications

- **Neural detector.** The provided model skeleton in graspcnn was trained on the Cornell Grasp Dataset. The network architecture was an encoder-decoder with channel progression 32-64-128-64-32. Additional preprocessing was introduced so that inference matched training conditions: RGB channels were normalized to [0,1] and zero-centered, while the depth channel was mean-centered and clipped. This preprocessing was important to stabilize RGB-D inference and reduce the gap between offline training and deployment.
- **Simulation integration.** The neural detector was integrated in p3_utils.py and grasp_pipeline.py, while preserving the circle detector as a geometric baseline. The simulator pipeline captured RGB-D images, predicted 2D grasps, converted them into 3D grasp targets, and executed pre-grasp, grasp, post-grasp, and lift motions.
- **Benchmark script.** grasp_evaluation.py was extended to evaluate five objects over ten trials each, hide non-target objects, seed random object placement for reproducibility, and compute per-object success rates.
- **Real-world pipeline.** grasp_object.py and GraspDetectorNN.py were modified for RealSense streaming, camera-to-robot transformation, RGB-D neural inference, gripper-width control, and full execution on the Airbot platform.

File	Role	Final modifications
graspcnn/train_model.py	Training	Used for all ablation runs; supports depth, RGB, or RGB-D input and saves model checkpoints with IoU scores.
DISCOVERSE/.../p3_utils.py	Simulator detector	Loaded the RGB-D model, added matching preprocessing, and enabled detector switching between circle and network modes.
DISCOVERSE/scripts_p3/grasp_evaluation.py	Simulation evaluation	Added seeded multi-object evaluation, object hiding, grasp execution, and per-object statistics.
GraspDetectorNN.py + grasp_object.py	Real deployment	Enabled RGB-D inference, depth-based 3D grasp conversion, hand-eye calibration, and Airbot execution with RealSense input.

2. Training setup and model selection

All training runs used CornellDataset with a 90/10 train-validation split, random rotation, random zoom, and selectable input modalities. The baseline network followed an encoder-decoder structure with feature dimensions 32-64-128-64-32, for a total of 447,172 trainable parameters. In addition, preprocessing was explicitly aligned between training and inference: RGB channels were normalized and zero-centered, and the depth channel was mean-centered and clipped. Several runs were then compared to assess input modality, model capacity, and training duration.

Experiment	Input	Configuration	Observed result	Decision
Depth-only baseline	Depth	bs=12, 100 epochs, 40 batches/epoch, 5 val batches	Validation improved late but grasping remained clearly weaker than RGB-D.	Rejected
RGB-D baseline	RGB-D	bs=12, 100 epochs, 40	Repeated 1.00 validation IoU	Selected

		batches/epoch, 5 val batches	checkpoints; training time ≈10 min.	
Large RGB-D model	RGB-D	64-128-256 channels, bs=10, 100 epochs, 40 batches/epoch	1.77 M parameters, slower training (≈15.6 min) without clear deployment benefit.	Not selected
Long RGB-D run	RGB-D	bs=12, 150 epochs, 60 batches/epoch, 10 val batches, 6 workers	Repeated 1.00 validation IoU again, but no robust improvement over baseline; ≈21.4 min.	Useful ablation

Selected model: the RGB-D baseline checkpoint epoch_90_iou_1.00 was retained for integration and deployment because it was simple, stable, and already reached the best validation score.

3. Example network output and simulation evaluation

Figure 1 shows a typical RGB-D neural output. The detector successfully placed the grasp rectangle on the object, but the quality map still exhibited edge artifacts near image borders. This motivated border suppression on the score map and candidate filtering during inference.

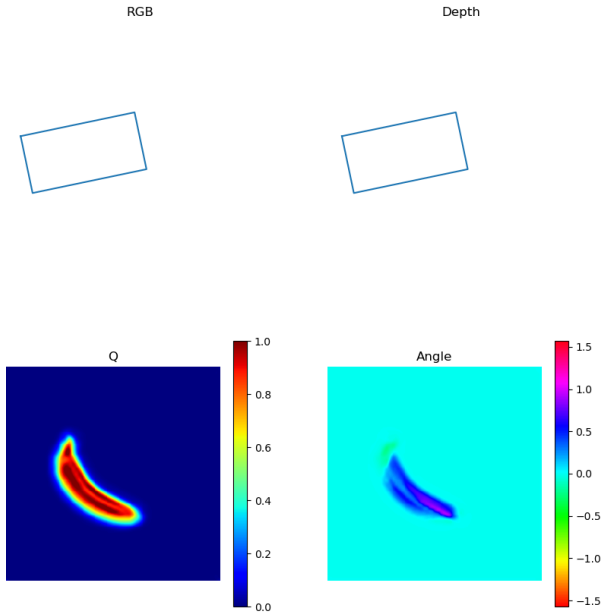


Figure 1. Example RGB-D neural grasp output and quality map.

The final uploaded evaluation script tested five objects over ten trials each. Using the terminal output provided in the attached text file, the overall simulation result was 39 successful grasps out of 50 attempts, corresponding to a 78% success rate. DISCOVERSE printed a GLIBC-related gaussian-renderer warning at launch, but this warning was non-fatal: the MuJoCo evaluation scene still loaded and the benchmark completed normally. In the final run, remaining failures were limited to two categories: inverse-kinematics exceptions on a small number of extreme poses and unsuccessful lifts when the gripper reached the object but did not raise it above the success threshold.

Object	Success rate	Comment
banana	0.60	Six of ten trials succeeded; the remaining failures were two IK exceptions and two weak lifts.
lemon	0.70	Seven successful grasps out of ten; failures were caused by insufficient lift rather than systematic misdetection.
mug	0.90	Best final result with nine successful grasps; only one attempt failed because the object was not lifted enough.
screwdriver	0.80	Eight successful grasps; one IK exception and one insufficient lift

		remained for this thin geometry.
sugar_box	0.90	Nine successful grasps; one trial failed to lift high enough while all other attempts succeeded.

4. Real-world deployment and discussion

After the simulation pipeline was stabilized, the neural detector was deployed on the real Airbot arm with a RealSense depth camera. The complete procedure included camera streaming, point labeling, hand–eye calibration, connection to the Airbot server, and execution of the grasp sequence on the physical robot. In contrast with simulation, the real tests were strongly affected by calibration tolerance, pose execution error, and the geometric mismatch between the predicted grasp rectangle and the actual gripper closing direction.

Five representative objects were tested in real conditions, with three trials per object: a mint box placed flat, a mint box placed vertically, a roll of tape, a pen, and a ball. The observed success rates were 0/3 for the flat mint box, 1/3 for the vertical mint box, 2/3 for the tape roll, 0/3 for the pen, and 1/3 for the ball. This corresponds to an overall real-world success rate of 4/15 = 26.7%, which is much lower than the 78% simulation success rate and clearly shows the remaining sim-to-real gap.

flat mint box and pen (up) tape and ball (down)

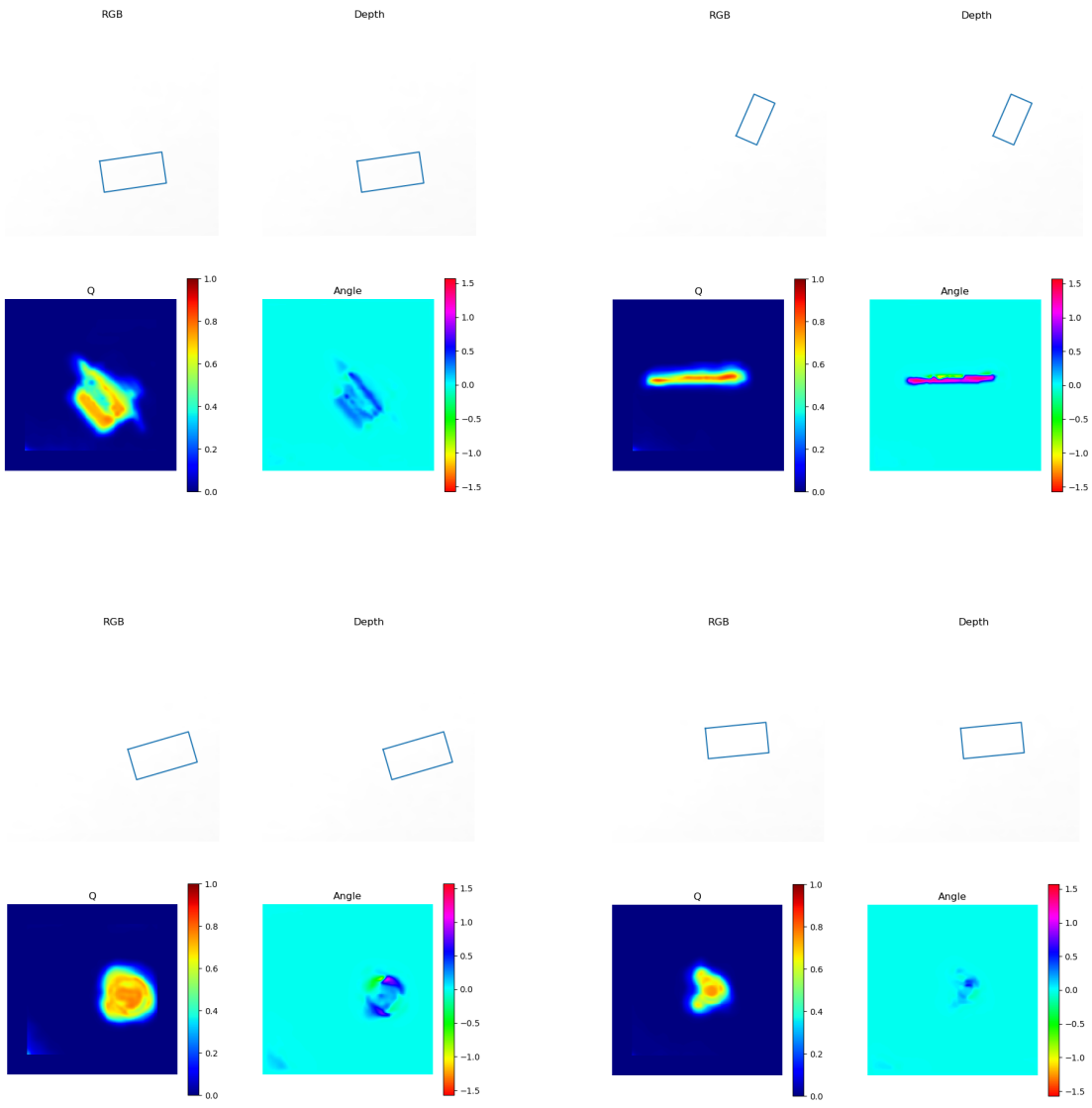


Figure 2 Representative real-world neural outputs collected during physical testing

The examples include flat and vertical mint boxes, tape, pen, and ball grasps. In several cases the quality map correctly highlighted the object, but the final executed grasp was still degraded by pose bias, angle mismatch, or border artifacts. In order to improve robustness, additional preprocessing was introduced on the RGB-D input and on the predicted grasp quality map, notably to reduce spurious responses near the borders and stabilize the grasp selection.

The dominant failure mode was inaccurate aiming of the robot, especially for objects placed closer to image corners. Even when the quality map localized the object reasonably well, a small translation error in the projected grasp center could shift the gripper enough to miss the object or to approach it off-center. This issue was particularly damaging for narrow objects such as the pen and for the flat mint box, where a few millimeters of bias were enough to make the grasp fail completely.

A second recurrent issue was incorrect grasp orientation. In several failed trials, the grasp rectangle angle was plausible in the image, but one jaw of the gripper would contact the object before the end-effector had fully aligned and closed. This explains why the tape roll and the ball still achieved occasional successes while the pen performed poorly: elongated or thin objects require a much more accurate angular estimate than bulkier shapes.

A third issue was the persistent parasitic response in the lower-left corner of the quality map. Border clipping was introduced to suppress those artifacts, and it improved the plots, but the fix remained only partially effective. The remaining corner response occasionally competed with the true object response and reduced the robustness of the selected grasp, especially for small targets and scenes with weak depth contrast.

In addition, the real robot showed an almost systematic vertical offset of the end-effector during execution. To compensate for this bias, the target grasp position and the grasp depth were manually adjusted in the control pipeline. More precisely, the target position was shifted vertically, and the effective grasp depth was reduced so that the robot approached the object from a slightly safer height. These modifications improved the stability of the real grasps and reduced the number of failures caused by the gripper descending too low before closure.

These real experiments show that the RGB-D neural model is able to produce meaningful grasp proposals, but that end-to-end success on hardware still depends on precise calibration, reliable border suppression, suitable preprocessing, and a grasp angle that is compatible with the physical jaw geometry. The real tests therefore validate the pipeline qualitatively while also identifying the main limitations that must be addressed to improve physical grasp reliability.

Conclusion

The project validated an end-to-end learning-based grasping pipeline that spans training, simulation, and real-world execution. The strongest experimental conclusion is that RGB-D grasp prediction clearly outperformed depth-only input. The main limitations at the end of the project were not purely network accuracy, but system-level issues: IK reachability in simulation, border artifacts in the score map, and sensitivity of the real robot to initialization and grasp height. Even with these limitations, the final system demonstrated a working perception-to-action manipulation pipeline and provided a clear basis for further improvement.